

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. Consider the bubble sort algorithm applied to an array of 1 / 1 point

- n
- elements. What is the time complexity of bubble sort in the average-case scenario?
- ☒

$O(n^2)$
- ☐

$O(n^3)$
- ☐

$O(n)$
- ☐

$O(n \log n)$

Nice work

Great job! Bubble sort has an average time complexity of

$$O(n^2)$$

because it repeatedly traverses the list, comparing adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the list is sorted.

2. Which of the following best describes the purpose of a greedy algorithm in algorithm design? 1 / 1 point

- ☐

To use dynamic programming to store and reuse previously computed results
- ☒

To find a global optimum by making locally optimal choices at each step
- ☐

To divide a problem into subproblems, solve each one, and combine the results
- ☐

To explore all possible solutions and find the best one

Nice work

Correct! A greedy algorithm aims to find a global optimum by making locally optimal choices at each step. This approach does not always yield the optimal solution for all problems, but it is efficient for specific cases.

3. Explain the mechanics of the quick sort algorithm. 1 / 1 point

- ☒

Quick sort uses a divide-and-conquer approach to sort elements by selecting a pivot and partitioning the array.
- ☐

Quick sort sequentially compares each pair of elements and swaps them if needed.
- ☐

Quick sort builds a min-heap to efficiently retrieve the smallest element.
- ☐

Quick sort sorts elements by recursively merging sorted halves.

Nice work

Excellent! Quick sort indeed uses a divide-and-conquer strategy, making it efficient for large datasets.

4. Which of the following sorting algorithms has the best average-case time complexity in sorting a randomly ordered array of n distinct numbers? 1 / 1 point

- ☐

Selection Sort
- ☒

Merge Sort
- ☐

Insertion Sort
- ☐

Bubble Sort

Nice work

Correct! Merge Sort has an average-case time complexity of

$$O(n \log n)$$

. Congratulations on understanding sorting algorithms!

5. What is the significance of Big O notation in analyzing the growth of functions? 1 / 1 point

- ☐

Big O notation provides the exact runtime of an algorithm.
- ☐

Big O notation measures the average growth rate of functions.
- ☒

Big O notation categorizes algorithms based on their worst-case growth rates.
- ☐

Big O notation defines the best-case scenario for algorithm performance.

Nice work

Correct! Big O notation is used to describe the upper bound or worst-case scenario for the growth rate of an algorithm.